

ATDD Workflow for Agentic AI

PRACTICAL GUIDE & EXECUTABLE SPECIFICATION EXAMPLE

Core Concept: Acceptance Test-Driven Development (ATDD) acts as a stable, deterministic "fitness function" for non-deterministic AI agents. Instead of reviewing complex generated code line-by-line, the human programs the behavioral contract, and the AI agent self-verifies against it.

1. The Human Input (The Executable Specification)

Instead of prompting an AI agent with ambiguous instructions (e.g., *"Write code to filter user accounts"*), the engineer provides a precise, human-readable specification using industry-standard **Gherkin syntax**.

```
# file: feature_flags.feature
```

```
Feature: Premium Feature Access
  As a system administrator
  I want to restrict advanced analytics to premium tier accounts
  So that we can monetize high-value features

  Scenario: Premium user accesses advanced analytics
    Given a user account with tier "Premium"
    When the user requests access to "Advanced Analytics"
    Then the system should "Allow" the request

  Scenario: Standard user is denied advanced analytics
    Given a user account with tier "Standard"
    When the user requests access to "Advanced Analytics"
    Then the system should "Deny" the request
```

2. The Agentic Prompt Loop

The engineer instructs the AI agent to use this precise file to govern its development lifecycle, forcing a disciplined **Red-Green-Refactor** inner loop.

System Prompt to the AI Agent:

```
"You are an agentic software developer. Read `feature_flags.feature`.
1. Generate the step definitions (the test plumbing).
2. Run the tests and confirm they fail (Red).
3. Write the minimal implementation code to pass the tests (Green).
4. Do not output final results until all tests pass successfully."
```

3. The AI's Generated Code & Passing Loop

The AI agent interprets the criteria, binds them to a test framework (like Python's behave), and iterates in its local sandbox until the tests pass.

file: steps/security_steps.py

```
from behave import given, when, then

class SecuritySystem:
    def check_access(self, tier, feature):
        if feature == "Advanced Analytics" and tier != "Premium":
            return "Deny"
        return "Allow"

@given('a user account with tier "{tier}"')
def step_impl(context, tier):
    context.user_tier = tier

@when('the user requests access to "{feature}"')
def step_impl(context, feature):
    system = SecuritySystem()
    context.result = system.check_access(context.user_tier, feature)

@then('the system should "{expected_action}" the request')
def step_impl(context, expected_action):
    assert context.result == expected_action
```

Conclusion

When working this way, the generated implementation code is treated similarly to how traditional developers treat compiler-generated assembly language. High confidence is achieved not through tedious manual code inspection, but through strict automated evaluation against the defined behavioral specification.

Based on Modern Software Engineering AI Briefing series.